1. ACT： 算法 / 模型逻辑

ACT模型 | 斯坦福ALOHA团队 | Action Chunking Transformer
致力于**低价值**硬件发挥出**高精度**操作的性能

不管用什么模型都是经历下面的循环

下面省略了一些细节代码，只保留核心部分

```
@safe_stop_image_writer
def record_loop():
    """
    核心控制循环：负责观测获取、动作推理、指令发送及数据保存。
    该循环体现了：观测 -> 推理 -> 动作 -> 执行 -> 等待 的完整链路。
    """
    # 检查数据集帧率与请求帧率是否匹配，确保控制频率一致性
    if dataset is not None and dataset.fps != fps:
        raise ValueError(f"The dataset fps should be equal to requested fps ({datase

    # 如果提供了策略模型（如 ACT），在开始前重置其内部状态（如 Transformer 的缓存）
    if policy is not None:
        policy.reset()

    timestamp = 0
    start_episode_t = time.perf_counter() # 记录回合开始的绝对时间

    # 主循环：在指定的回合时间内持续运行
    while timestamp < control_time_s:
        start_loop_t = time.perf_counter() # 记录本帧循环的开始时间

        # 检查是否通过键盘等事件触发了提前退出
        if events["exit_early"]:
            events["exit_early"] = False
            break

        # --- ① 获取观测 (Observation) ---
        # 链路：机器人硬件 -> 内核驱动缓冲区 -> Python dict
        observation = robot.get_observation()

        # 如果需要保存数据或执行策略，将原始观测转换为数据集定义的特征格式
```

```
        if policy is not None or dataset is not None:
            observation_frame = build_dataset_frame(dataset.features, observation, p

        # --- ② 推理/获取动作 (Action Generation) ---
        if policy is not None:
            # 自主模式: 调用神经网络模型 (如 ACT) 预测动作值
            action_values = predict_action(
                observation_frame,
                policy,
                get_safe_torch_device(policy.config.device),
                policy.config.use_amp,
                task=single_task,
                robot_type=robot.robot_type,
            )
            # 将张量结果映射回机器人的动作特征键值对
            action = {key: action_values[i].item() for i, key in enumerate(robot.act

        # --- ③ 动作发送 (Send Action) ---
        # 链路: Python -> os.write -> TTY/ACM -> USB Controller (DMA) -> 机器人控制器
        # 注意: 此处 send_action 在数据提交给内核/硬件队列后即返回, 与物理传输并行
        sent_action = robot.send_action(action)

        # --- 数据记录 (Logging) ---
        if dataset is not None:
            # 构建动作帧并将其与观测帧合并, 存入数据集缓冲区
            action_frame = build_dataset_frame(dataset.features, sent_action, prefix
            frame = {**observation_frame, **action_frame}
            dataset.add_frame(frame, task=single_task)
        # --- ④ 频率对齐 (Busy Wait) ---
        # 计算本轮逻辑消耗的时间
        dt_s = time.perf_counter() - start_loop_t
        # 忙等: 等待直到填满 1/fps 的时长, 确保控制频率稳定, --dataset.fps int: Limit the fr
        # 此时硬件正在通过 DMA 并行发送动作包
        busy_wait(1 / fps - dt_s)
        # 更新回合已持续的时间
        timestamp = time.perf_counter() - start_episode_t
```

观测->推理->动作->执行->...的循环

FPS如果大的话有可能会达到USB发送->推理->发送的循环的极限

感觉可以从**提升USB发送速度->提升FPS->提升模型速度**来对整体进行优化

```
def predict_action():
    # 浅拷贝观测数据，避免修改原始数据源
    observation = copy(observation)

    with (
        # 开启推理模式：禁用梯度计算，减少内存消耗并加速计算
        torch.inference_mode(),
        # 如果使用 CUDA 且开启了混合精度(AMP)，则启用自动混合精度上下文
        torch.autocast(device_type=device.type) if device.type == "cuda" and use_amp
    ):
        # 遍历所有观测特征（如图像、关节角度等），将其转换为 PyTorch 格式
        for name in observation:
            # 将 NumPy 数组转换为 PyTorch 张量
            observation[name] = torch.from_numpy(observation[name])

            # 针对图像数据的特殊预处理：
            if "image" in name:
                # 归一化：将像素值从 [0, 255] 映射到 [0, 1.0] 的 float32 类型
                observation[name] = observation[name].type(torch.float32) / 255
                # 维度转换：将 (H, W, C) 转换为 PyTorch 要求的 (C, H, W) 格式
                observation[name] = observation[name].permute(2, 0, 1).contiguous()

            # 增加批次维度 (Batch Dimension)，将形状变为 (1, C, H, W)
            observation[name] = observation[name].unsqueeze(0)
            # 将数据移动到计算设备（如 CPU、CUDA 或 MPS）
            observation[name] = observation[name].to(device)

        # 注入任务描述和机器人类型信息（用于多任务策略）
        observation["task"] = task if task else ""
        observation["robot_type"] = robot_type if robot_type else ""

        # --- 核心推理阶段 ---
        # 调用策略模型的 select_action 方法，根据当前观测计算动作
        # 对于 ACT 模型，这会通过 Transformer 的编码器-解码器生成动作序列
        action = policy.select_action(observation)

        # 移除推理结果中的批次维度
        action = action.squeeze(0)
```

```
        # 将生成的动作移动回 CPU 内存，以便后续通过串口(os.write)发送给硬件
        action = action.to("cpu")

    return action
```

模型在 `lerobot/src/lerobot/policies/act/configuration_act.py` 和 `lerobot/src/lerobot/policies/act/modeling_act.py`

参考论文《使用低成本硬件学习细粒度双手动操作》（https://huggingface.co/papers/2304.13705）。

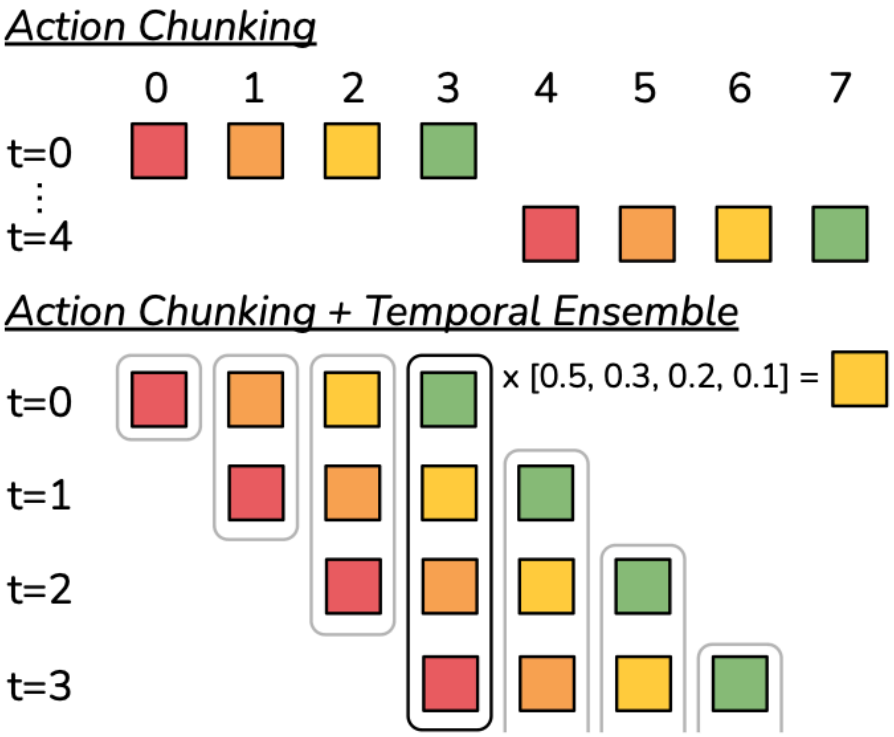

Fig. 5: We employ both Action Chunking and Temporal Ensembling when applying actions, instead of interleaving observing and executing.

1. 上半部分：Action Chunking（动作分块）

- 一次预测，一段动作（t=0的时候预测0-4的动作，t=4的时候预测4-8的动作）
- 固定间隔更新：机器人会依次执行这 4 个动作。直到 t=4 时，模型才会再次进行观测并预测下一个动作块。
- 局限性：这种方式虽然减少了推理频率，但在两个动作块的交界处（例如 t=3 和 t=4 之间）可能会出现动作不连贯或抖动。

2. 下半部分：Action Chunking + Temporal Ensemble（时序集成）

是 ACT 实现高精度和平滑度的核心考虑。它不再是每隔几步更新一次，而是**每一帧都更新，但利用重叠的预测**，其实是**用计算压力换动作平滑**了。

- 滑动窗口式的预测：
  - 在 t=0 时，预测一组序列。
  - 在 t=1 时，再次观测并预测一组新的序列。注意图中 t=1 的序列相对于 t=0 向右滑动了一格。
- 指数加权平均（加权求和）
  - 系统会给这些重叠的动作分配权重（如图中显示的 [0.5, 0.3, 0.2, 0.1]）。越早预测的动作权重越高，通过加权平均得到最终执行的动作。

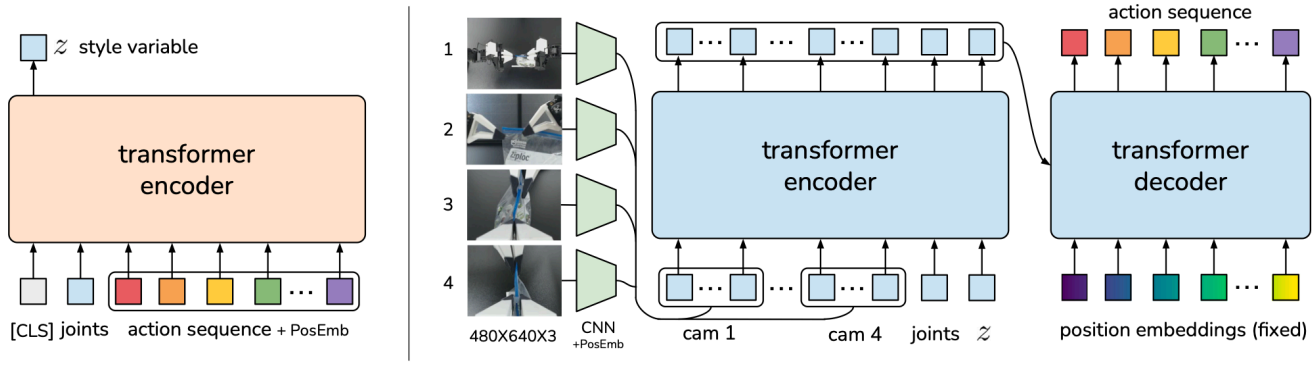

Fig. 4: *Architecture of Action Chunking with Transformers (ACT).* We train ACT as a Conditional VAE (CVAE), which has an encoder and a decoder. *Left:* The encoder of the CVAE compresses action sequence and joint observation into z, the style variable. The encoder is discarded at test time. *Right:* The decoder or policy of ACT synthesizes images from multiple viewpoints, joint positions, and z with a transformer encoder, and predicts a sequence of actions with a transformer decoder. z is simply set to the mean of the prior (i.e. zero) at test time.

1. 左侧：训练阶段的 CVAE 编码器 (Encoder)

- 输入：包含当前时刻的机器人**关节位置**（joints）和完整的**专家动作序列**（action sequence）。
- 输出：压缩成一个风格变量 z（style variable）。
- 这个编码器**仅在训练时使用**，推理的时候Encoder会被删除

2. 右侧：推理阶段的策略网络 (Decoder/Policy)

- 输入：多视角图像、关节状态、风格变量 z。
- 通过一个Transformer-Encoder和一个Decoder
- 输出：一次性预测出一个动作分块（action sequence/chunk），未来几步的。

Training

Step 1: sample data

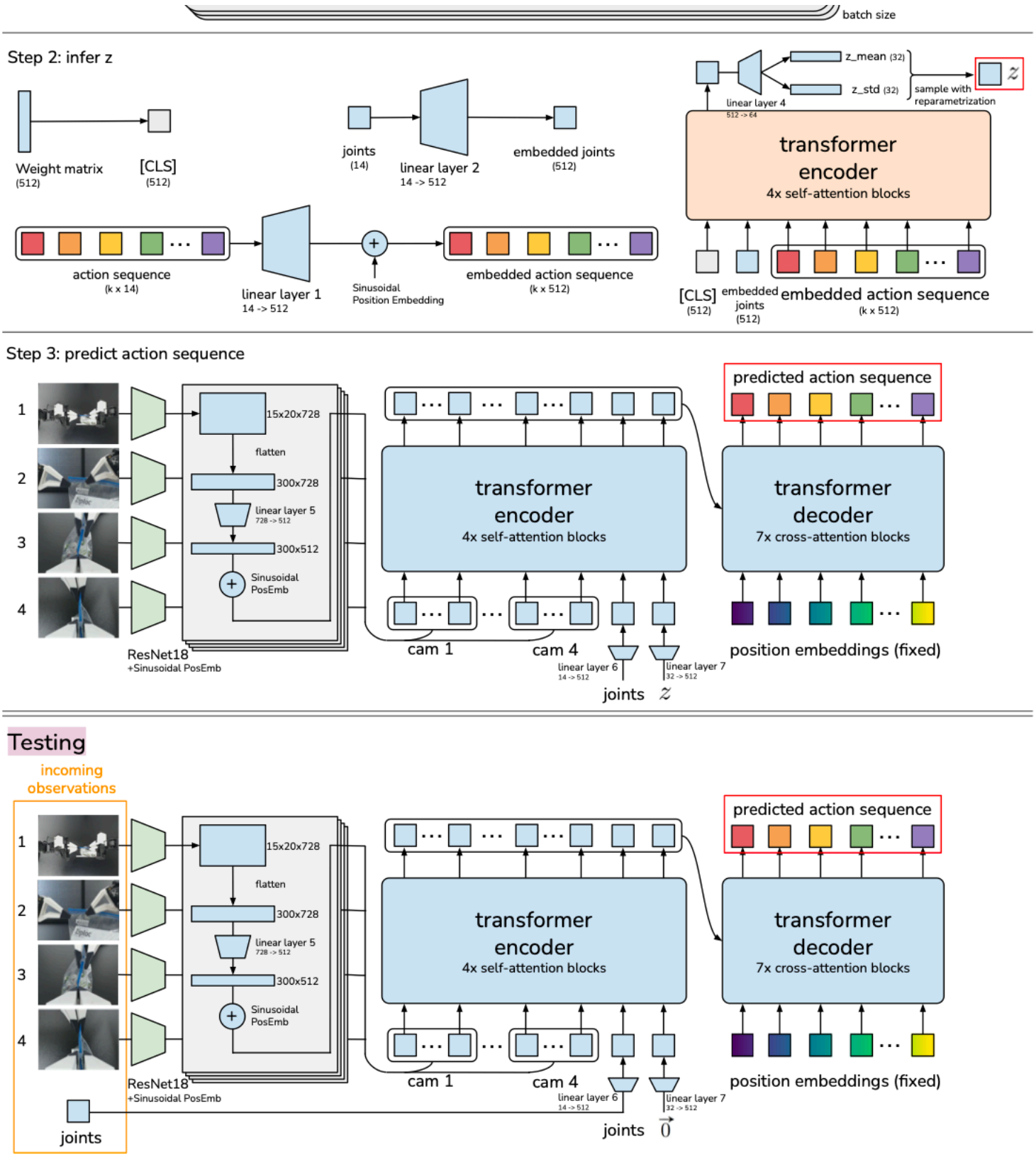

Fig. 11: Detail architecture of Action Chunking with Transformers (ACT).

训练

1. 训练集：图片+动作序列（本质是模仿学习）
2. 推理变量z：将动作序列和关节数据映射到隐藏维度（512维）。经过带有 4 个自注意力块的

Transformer 编码器，推导出 z 的均值和标准差，并通过重参数化技巧采样出 z。

3. 预测动作序列：图像通过 ResNet18 提取特征并平坦化（flatten）。视觉特征、关节数据和 z 一起进入策略的 Transformer 编码器。Transformer 解码器（带有 7 个交叉注意力块）输出预测的动作序列。

推理

- 与训练时的“预测”部分几乎一致，唯一区别是 z 直接设为 0。
- 模型根据当前的实时观测（图像和关节），一次性预测未来的一组动作序列。

这是其中一个模型，也可以换成Diffusion/Pi0等模型。

内核编译安装

现在是可以原生编译安装了

```
(lerobot) vel@raspberrypi:~ $ uname -a
Linux raspberrypi 6.12.61-MY_KERNEL+ #1 SMP PREEMPT Tue Dec 16 11:11:16 CST 2025 aarch64 GNU/Linux
(lerobot) vel@raspberrypi:~ $
```

内核评估

我问了AI

可以用系统原生的strace

```
(lerobot) vel@raspberrypi:~ $ strace -c -p 1632
strace: Process 1632 attached
^Cstrace: Process 1632 detached
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 79.57    0.084754        2922        29           wait4
  6.91    0.007358         253        29           clone
  2.89    0.003083         140        22           futex
  2.68    0.002855           3       859           epoll_pwait
  1.97    0.002098           3       528           read
  1.34    0.001423           5       274           openat
  1.14    0.001212           2       487           close
  0.67    0.000709           1       361           ioctl
  0.55    0.000581           1       324           setsockopt
  0.41    0.000437           1       264           newfstatat
  0.40    0.000422           5        81           socketpair
  0.33    0.000347           2       136           write
  0.24    0.000260           9        28           io_uring_enter
  0.23    0.000241           3        78        26 epoll_ctl
  0.18    0.000187           1       152           rt_sigprocmask
  0.16    0.000175           3        52           shutdown
  0.11    0.000112          12         9           getpid
  0.09    0.000098           3        27           pipe2
  0.06    0.000061           1        47           rt_sigaction
  0.05    0.000048           2        22           recvmsg
  0.03    0.000035           2        17           rt_sigreturn
  0.01    0.000009           1         5           kill
  0.01    0.000006           1         4           mprotect
  0.00    0.000003           1         2           madvise
  0.00    0.000000           0         3           fcntl
------ ----------- ----------- --------- --------- ----------------
100.00    0.106514          27      3840        26 total
```

可以看到系统调用的总时长，次数，占比等。